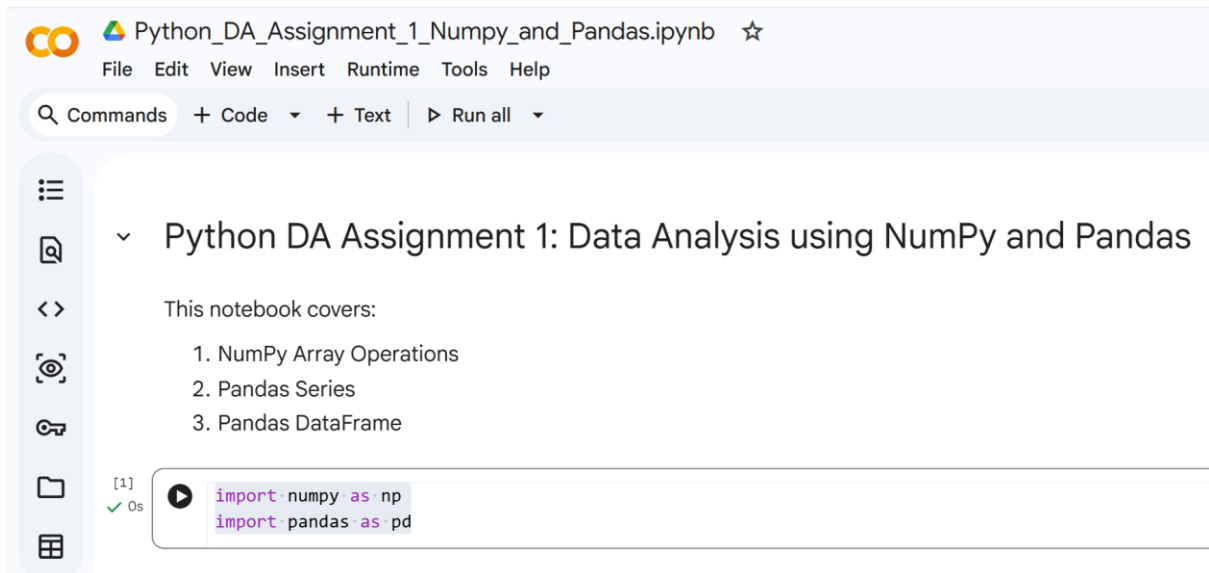


Python DA Assignment 1: Data Analysis using NumPy and Pandas

This notebook covers:

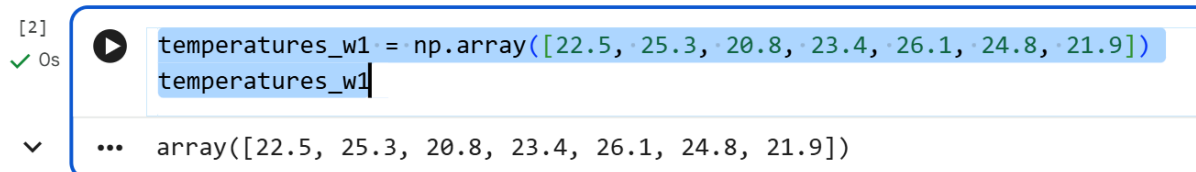
1. NumPy Array Operations
2. Pandas Series
3. Pandas DataFrame



1. NumPy Array Operations

1. NumPy Array Operations

1.1 Create a 1D NumPy Array (Week 1 temperatures)



✓ 1.2 Inspection and Properties

[3]
✓ 0s



```
print("Shape:", temperatures_w1.shape)
print("Data type:", temperatures_w1.dtype)
print("Number of elements:", temperatures_w1.size)
```



```
... Shape: (7,)
    Data type: float64
    Number of elements: 7
```

✓ 1.3 Array Operations

[4]
✓ 0s



```
# Convert Celsius to Fahrenheit
temperatures_w1_f = (temperatures_w1 * 9/5) + 32
print("Temperatures in Fahrenheit:", temperatures_w1_f)

# Max, min, mean
print("Max temperature:", temperatures_w1.max())
print("Min temperature:", temperatures_w1.min())
print("Mean temperature:", temperatures_w1.mean())
```



```
... Temperatures in Fahrenheit: [72.5  77.54 69.44 74.12 78.98 76.64 71.42]
    Max temperature: 26.1
    Min temperature: 20.8
    Mean temperature: 23.542857142857144
```

1.4 Array Slicing and Indexing

[5]
✓ Os



```
# First three days
first_three_days = temperatures_w1[0:3]
print("First three days:", first_three_days)

# Weekend (last two days)
weekend = temperatures_w1[-2:]
print("Weekend:", weekend)

# Middle three days (index 2,3,4)
middle_three_days = temperatures_w1[2:5]
print("Middle three days:", middle_three_days)
```

... First three days: [22.5 25.3 20.8]
Weekend: [24.8 21.9]
Middle three days: [20.8 23.4 26.1]

1.5 Create a 2D Array

[6]
✓ Os



```
temperatures_w2 = np.array([19.2, 22.5, 21.3, 24.0, 23.5, 22.8, 20.1])
temperatures = np.array([temperatures_w1, temperatures_w2])
temperatures
```

... array([[22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9],
[19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1]])

1.6 Inspect and Slice the 2D Array

[7]
✓ Os



```
print("Shape:", temperatures.shape)
print("Data type:", temperatures.dtype)
print("Total number of elements:", temperatures.size)
```

... Shape: (2, 7)
Data type: float64
Total number of elements: 14

[8]

✓ Os



```
# Temperatures for each week
week1 = temperatures[0]
week2 = temperatures[1]
print("Week 1 temperatures:", week1)
print("Week 2 temperatures:", week2)

# Weekends (last two days) for both weeks
weekends_both_weeks = temperatures[:, -2:]
print("Weekends for both weeks:\n", weekends_both_weeks)
```

✓ ... Week 1 temperatures: [22.5 25.3 20.8 23.4 26.1 24.8 21.9]
Week 2 temperatures: [19.2 22.5 21.3 24. 23.5 22.8 20.1]
Weekends for both weeks:
[[24.8 21.9]
[22.8 20.1]]

2. Pandas Series

✓ 2.1 Creating Pandas Series

[9]

✓ Os



```
marks = pd.Series([95, 92, 89, 85, 80],
                  index=['Rank1', 'Rank2', 'Rank3', 'Rank4', 'Rank5'])
marks
```

✓ ...

	0
Rank1	95
Rank2	92
Rank3	89
Rank4	85
Rank5	80

dtype: int64

✓ 2.2 Indexing and Slicing

[10]
✓ Os



```
# 1st rank student using integer index position
print("Mark of 1st rank (integer position):", marks.iloc[0])

# Top 3 ranks using loc
print("\nTop 3 ranks (loc):")
print(marks.loc[['Rank1', 'Rank2', 'Rank3']])

# 3rd ranked student using iloc
print("\nMark of 3rd rank (iloc):", marks.iloc[2])

# Boolean mask: marks greater than 90
print("\nRanks with marks greater than 90:")
print(marks[marks > 90])
```

✓

... Mark of 1st rank (integer position): 95

Top 3 ranks (loc):

Rank1 95

Rank2 92

Rank3 89

dtype: int64

Mark of 3rd rank (iloc): 89

Ranks with marks greater than 90:

Rank1 95

Rank2 92

dtype: int64

2.3 Manipulating Series

[11]
✓ Os



```
# Modify mark of 1st rank to 100
marks['Rank1'] = 100
print("After updating Rank1:\n", marks)

# Remove the last rank entry
marks = marks.drop('Rank5')
print("\nAfter removing Rank5:\n", marks)

# Compute CGPA by dividing each mark by 10
cgpa = marks / 10
print("\nCGPA:\n", cgpa)
```

✓

... After updating Rank1:

```
Rank1    100
Rank2     92
Rank3     89
Rank4     85
Rank5     80
dtype: int64
```

After removing Rank5:

```
Rank1    100
Rank2     92
Rank3     89
Rank4     85
dtype: int64
```

CGPA:

```
Rank1    10.0
Rank2     9.2
Rank3     8.9
Rank4     8.5
dtype: float64
```

3. Pandas DataFrame

3.1 Creating Pandas DataFrame

[12]
✓ Os

```
transactions = pd.DataFrame({  
    'TransactionID': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110],  
    'ProductCategory': ['Electronics', 'Clothing', 'Electronics', 'Furniture', 'Clothing',  
                        'Electronics', 'Furniture', 'Clothing', 'Furniture', 'Electronics'],  
    'Region': ['North', 'South', 'North', 'East', 'West', 'North', 'East', 'West', 'South', 'North'],  
    'Amount': [200, 150, 300, 450, 200, 250, 300, 180, 350, 400]  
})  
transactions
```

▼

	TransactionID	ProductCategory	Region	Amount
0	101	Electronics	North	200
1	102	Clothing	South	150
2	103	Electronics	North	300
3	104	Furniture	East	450
4	105	Clothing	West	200
5	106	Electronics	North	250
6	107	Furniture	East	300
7	108	Clothing	West	180
8	109	Furniture	South	350
9	110	Electronics	North	400

Next steps:

[Generate code with transactions](#)

[New interactive sheet](#)

3.2 Data Exploration

[13]

✓ Os



```
print("Head:\n", transactions.head())
print("\nTail:\n", transactions.tail())
print("\nShape:", transactions.shape)
print("\nColumn names:", transactions.columns.tolist())
print("\nData types:\n", transactions.dtypes)
print("\nInfo:")
transactions.info()
```

✓

... Head:

	TransactionID	ProductCategory	Region	Amount
0	101	Electronics	North	200
1	102	Clothing	South	150
2	103	Electronics	North	300
3	104	Furniture	East	450
4	105	Clothing	West	200

Tail:

	TransactionID	ProductCategory	Region	Amount
5	106	Electronics	North	250
6	107	Furniture	East	300
7	108	Clothing	West	180
8	109	Furniture	South	350
9	110	Electronics	North	400

Shape: (10, 4)

Column names: ['TransactionID', 'ProductCategory', 'Region', 'Amount']

Data types:

TransactionID	int64
ProductCategory	object
Region	object
Amount	int64

dtype: object

Info:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 10 entries, 0 to 9

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	TransactionID	10 non-null	int64
1	ProductCategory	10 non-null	object
2	Region	10 non-null	object
3	Amount	10 non-null	int64

dtypes: int64(2), object(2)

memory usage: 452.0+ bytes

[14]

✓ Os



```
# ProductCategory and Amount columns only  
transactions[['ProductCategory', 'Amount']]
```



ProductCategory Amount



0	Electronics	200
1	Clothing	150
2	Electronics	300
3	Furniture	450
4	Clothing	200
5	Electronics	250
6	Furniture	300
7	Clothing	180
8	Furniture	350
9	Electronics	400

[15]

✓ 0s



```
# Last 3 columns using iloc
transactions.iloc[:, -3:]
```



...

ProductCategory Region Amount



0	Electronics	North	200
1	Clothing	South	150
2	Electronics	North	300
3	Furniture	East	450
4	Clothing	West	200
5	Electronics	North	250
6	Furniture	East	300
7	Clothing	West	180
8	Furniture	South	350
9	Electronics	North	400

[16]

✓ 0s



```
# Filter rows where Region is 'North' and Amount > 200
north_high_amount = transactions[(transactions['Region'] == 'North') & (transactions['Amount'] > 200)]
north_high_amount
```



...

	TransactionID	ProductCategory	Region	Amount
2	103	Electronics	North	300
5	106	Electronics	North	250
9	110	Electronics	North	400



Next steps: [Generate code with north_high_amount](#)

[New interactive sheet](#)

How can I help you today?

Calculate the mean amount spent per product category

Plot a bar chart of total amount per region

Group transactions by ProductCategory and show descriptive statistics

transactions X north_high_amount X

What can I help you build?

+ ≡

Gemini 2.5 Flash ▾ ▶

[17]

✓ 0s



```
# Value counts for ProductCategory
transactions['ProductCategory'].value_counts()
```

▼

...

count

ProductCategory

Electronics	4
Clothing	3
Furniture	3

dtype: int64

[18]
✓ 0s

```
# Unique values in Region  
transactions['Region'].unique()
```



```
array(['North', 'South', 'East', 'West'], dtype=object)
```

[19]
✓ 0s



```
# Group by Region and find mean Amount  
transactions.groupby('Region')['Amount'].mean()
```



...

Amount

Region

East	375.0
------	-------

North	287.5
-------	-------

South	250.0
-------	-------

West	190.0
------	-------

dtype: float64

[20]

✓ 0s



```
# Modify Amount for TransactionID 102 to 165
```

```
transactions.loc[transactions['TransactionID'] == 102, 'Amount'] = 165  
transactions
```



	TransactionID	ProductCategory	Region	Amount
0	101	Electronics	North	200
1	102	Clothing	South	165
2	103	Electronics	North	300
3	104	Furniture	East	450
4	105	Clothing	West	200
5	106	Electronics	North	250
6	107	Furniture	East	300
7	108	Clothing	West	180
8	109	Furniture	South	350
9	110	Electronics	North	400



Next steps:

[Generate code with transactions](#)

[New interactive sheet](#)

GIVE ME TRANSACTION ID DETAILS OF 105?



transactions



north_high_amount

Here are the details for Transaction ID 105:



[20]

✓ Os



```
# Modify Amount for TransactionID 102 to 165
transactions.loc[transactions['TransactionID'] == 102, 'Amount'] = 165
transactions
```

✓

...

	TransactionID	ProductCategory	Region	Amount
0	101	Electronics	North	200
1	102	Clothing	South	165
2	103	Electronics	North	300
3	104	Furniture	East	450
4	105	Clothing	West	200
5	106	Electronics	North	250
6	107	Furniture	East	300
7	108	Clothing	West	180
8	109	Furniture	South	350
9	110	Electronics	North	400



Next steps:

[Generate code with transactions](#)

[New interactive sheet](#)

★ Gemini

[]

```
transactions[transactions['TransactionID'] == 105]
```

```
[ ] # Remove the row with TransactionID 109
transactions = transactions[transactions['TransactionID'] != 109].reset_index(drop=True)
transactions
```

	TransactionID	ProductCategory	Region	Amount	Discount
0	101	Electronics	North	200	20.0
1	102	Clothing	South	165	16.5
2	103	Electronics	North	300	30.0
3	104	Furniture	East	450	45.0
4	105	Clothing	West	200	20.0
5	106	Electronics	North	250	25.0
6	107	Furniture	East	300	30.0
7	108	Clothing	West	180	18.0
8	110	Electronics	North	400	40.0

```
[ ] # Gemini
furniture_transactions = transactions[transactions['ProductCategory'] == 'Furniture']
display(furniture_transactions)
```

Gemini

Variables X

⋮ X

Name	Type	Shape	Value
cgpa	Series	(4,)	Rank1 10.0 Rank2 9.2 Rank3 8.9 Rank4 8.5 dtype: float64
first_three_da...	ndarray	(3,)	array([22.5, 25.3, 20.8])
marks	Series	(4,)	Rank1 100 Rank2 92 Rank3 89 Rank4 85 dtype: int64
middle_three_...	ndarray	(3,)	array([20.8, 23.4, 26.1])
north_high_a...	DataFrame	(3, 4)	TransactionID ProductCategory Region Amount 2 103 Electronics North 300 5 106 Ele
temperatures	ndarray	(2, 7)	array([[22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9], [19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1]])
temperatures...	ndarray	(7,)	array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])
temperatures...	ndarray	(7,)	array([72.5, 77.54, 69.44, 74.12, 78.98, 76.64, 71.42])
temperatures...	ndarray	(7,)	array([19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1])
transactions	DataFrame	(10, 5)	TransactionID ProductCategory Region Amount Discount 0 101 Electronics North 200
week1	ndarray	(7,)	array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])
week2	ndarray	(7,)	array([19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1])
weekend	ndarray	(2,)	array([24.8, 21.9])
weekends_bo...	ndarray	(2, 2)	array([[24.8, 21.9], [22.8, 20.1]])

[25]
✓ 0s

```
# Delete the Discount column
transactions = transactions.drop(columns=['Discount'])
transactions
```

✓

	TransactionID	ProductCategory	Region	Amount
0	101	Electronics	North	200
1	102	Clothing	South	165
2	103	Electronics	North	300
3	104	Furniture	East	450
4	105	Clothing	West	200
5	106	Electronics	North	250
6	107	Furniture	East	300
7	108	Clothing	West	180
8	109	Furniture	South	350
9	110	Electronics	North	400

Next steps: [Generate code with transactions](#) [New interactive sheet](#)

◆ Gemini

[]

```
clothing_transactions = transactions[transactions['ProductCategory'] == 'Clothing']
display(clothing_transactions)
```

Gemini ✕

Variables ✕

⋮ ✕

Name	Type	Shape	Value
cgpa	Series	(4,)	Rank1 10.0 Rank2 9.2 Rank3 8.9 Rank4 8.5 dtype: float64
first_three_da...	ndarray	(3,)	array([22.5, 25.3, 20.8])
marks	Series	(4,)	Rank1 100 Rank2 92 Rank3 89 Rank4 85 dtype: int64
middle_three_...	ndarray	(3,)	array([20.8, 23.4, 26.1])
north_high_a...	DataFrame	(3, 4)	TransactionID ProductCategory Region Amount 2 103 Electronics North 300 5 106 Ele
temperatures	ndarray	(2, 7)	array([[22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9], [19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1]])
temperatures...	ndarray	(7,)	array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])
temperatures...	ndarray	(7,)	array([72.5 , 77.54, 69.44, 74.12, 78.98, 76.64, 71.42])
temperatures...	ndarray	(7,)	array([19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1])
transactions	DataFrame	(10, 4)	TransactionID ProductCategory Region Amount 0 101 Electronics North 200 1 102 Clo
week1	ndarray	(7,)	array([22.5, 25.3, 20.8, 23.4, 26.1, 24.8, 21.9])
week2	ndarray	(7,)	array([19.2, 22.5, 21.3, 24. , 23.5, 22.8, 20.1])
weekend	ndarray	(2,)	array([24.8, 21.9])
weekends_bo...	ndarray	(2, 2)	array([[24.8, 21.9], [22.8, 20.1]])

[26]
✓ Os



```
display(transactions)
```



	TransactionID	ProductCategory	Region	Amount
0	101	Electronics	North	200
1	102	Clothing	South	165
2	103	Electronics	North	300
3	104	Furniture	East	450
4	105	Clothing	West	200
5	106	Electronics	North	250
6	107	Furniture	East	300
7	108	Clothing	West	180
8	109	Furniture	South	350
9	110	Electronics	North	400



Next steps:

[Generate code with transactions](#)[New interactive sheet](#)